

1 Inferred function types

다음 코드에 대하여 질문에 답하십시오.

```
class Centimeter(val value: Double) {  
    fun plus(other: Centimeter): Centimeter =  
        Centimeter(value + other.value)  
  
    fun times(other: Double): Centimeter =  
        Centimeter(value * other)  
  
    override fun toString(): String = "$value cm"  
}  
  
val Int.cm get() = Centimeter(this.toDouble())  
  
fun distance(from: Centimeter, to: Centimeter): Centimeter =  
    Centimeter(abs(to.value - from.value))
```

아래의 함수 참조에 대해, 결과로 나올 함수의 타입을 예측해 보시오.

1. Centimeter::plus
2. Centimeter::times
3. Centimeter::value
4. Centimeter::toString
5. Centimeter(1.0)::plus
6. Centimeter(2.0)::times
7. Centimeter(3.0)::value
8. Centimeter(4.0)::toString
9. Int::cm
10. 123::cm
11. ::distance

번호	함수 참조	함수의 타입
1	Centimeter::plus	(Centimeter, Centimeter) -> Centimeter
2	Centimeter::times	(Centimeter, Double) -> Centimeter
3	Centimeter::value	(Centimeter) -> Double
4	Centimeter::toString	(Centimeter) -> String
5	Centimeter(1.0)::plus	(Centimeter) -> Centimeter
6	Centimeter(2.0)::times	(Double) -> Centimeter
7	Centimeter(3.0)::value	() -> Double
8	Centimeter(4.0)::toString	() -> String
9	Int::cm	(Int) -> Centimeter
10	123::cm	() -> Centimeter
11	::distance	(Centimeter, Centimeter) -> Centimeter

함수의 타입이 정해지는 이유

1. 특정 객체가 정해지지 않았기 때문에 함수를 호출할 때 사용할 Centimeter객체가 첫 번째 매개변수로 추가

객체 + Centimeter -> Centimeter

2. 객체 + Double -> Centimeter

3. 객체 -> Double

4. 객체 -> String

5. 함수를 실행할 객체가 이미 정해짐. 따라서 수신 객체를 따로 받을 필요가 없음

원래 매개변수 -> 반환값

Centimeter -> Centimeter

6. Double -> Centimeter

7. 입력 없음 -> Double

8. 입력 없음 -> String

9. cm는 Int 객체를 사용하는 확장 프로퍼티. 어떤 정수인지 정해지지 않았으므로 Int를 입력받음.

(Int) -> Centimeter

10. 정수가 123으로 이미 정해짐. 따라서 입력 없이 결과만 반환

() -> Centimeter

11. distance는 클래스에 속한 멤버 함수가 아닌 일반함수. 따라서 수신 객체를 추가하지 않고 원래 매개변수와 반환타입을 그대로 사용